

CSCE 647 IMAGE SYNTHESIS · MODULE 06 · LECTURE NOTES

Sampling and Variance Reduction

Choosing where to look, which is the whole game.

MODULE OUTCOMES

- Derive and apply the inverse CDF method.
- Sample the hemisphere uniformly and proportionally to the cosine.
- Explain why importance sampling reduces variance and when it fails.
- Compare stratification, Latin hypercube, and low-discrepancy sequences.
- Apply Russian roulette without introducing bias.

1 Drawing from a distribution

A random number generator gives uniform variates on $[0,1)$. Every sampling routine in a renderer is a method for turning those into something else.

$$P(x) = \int_{-\infty}^x p(t) dt \quad x = P^{-1}(\xi), \quad \xi \sim U[0,1)$$

The cumulative distribution function is monotonic and runs from 0 to 1, so it can be inverted. Feeding a uniform variate into the inverse produces a sample distributed according to p . Geometrically, the CDF is steep where the density is high, so a uniform step in ξ maps to a small step in x there — which is precisely concentrating samples where the density is large.

1.1 Cosine-weighted hemisphere sampling

The most useful instance in rendering. The rendering equation contains a $\cos\theta$ factor, so sampling proportionally to the cosine removes it from the estimator entirely.

```
vec3 cosine_sample_hemisphere(float u1, float u2) {
    float r = sqrtf(u1);           // sqrt: uniform area on the disk
    float phi = 2.0f * PI * u2;
    float z = sqrtf(fmaxf(0.0f, 1.0f - u1));
    return vec3(r*cosf(phi), r*sinf(phi), z);
}
float cosine_pdf(const vec3& w) { return w.z / PI; }
```

This is **Malley's method**: sample a point uniformly on the unit disk and project it vertically onto the hemisphere. The result is exactly cosine-distributed, with no trigonometric inversion required. The $\sqrt{}$ in the radius is what makes the disk sample uniform by *area* — using $u1$ directly clusters samples near the centre, and it is the most common bug in this routine.

WHY THIS PARTICULAR DENSITY IS WORTH THE TROUBLE

Take a Lambertian surface with albedo ρ . The integrand is $(\rho/\pi) \cdot L \cdot \cos\theta$, and the cosine-weighted density is $\cos\theta/\pi$. The estimator is their ratio:

$$f/p = (\rho/\pi \cdot L \cdot \cos\theta) / (\cos\theta/\pi) = \rho \cdot L$$

The cosine cancels and the π cancels. A diffuse bounce reduces to multiplying the path throughput by the albedo — one multiply, no trigonometry, and **zero variance contributed by the cosine factor**.

This is importance sampling working perfectly on one factor of the integrand, and it is free. It also explains why diffuse surfaces are the cheapest thing a path tracer renders.

2 Importance sampling

VARIANCE COMES FROM THE RATIO, NOT THE FUNCTION

The Monte Carlo estimator returns $f(X)/p(X)$. Its variance is the variance of *that ratio* across samples.

If f/p were constant, every sample would return the same number and the variance would be exactly zero — regardless of how complicated f itself is.

So the goal is to choose p proportional to f . Where the integrand is large, sample often; where it is small, sample rarely; and the division by p undoes the bias this introduces.

The ideal $p = f/|f|$ gives zero variance and is unobtainable, because normalising it requires the integral we set out to compute. **But the integrand is a product, and its factors can be matched individually.**

Factor of the integrand	Sampled by	Covered in
$\cos\theta_i$	Malley's method. Exact, and free.	This module.
The BRDF f	Analytic inversion of each material's lobe — GGX has a closed-form visible-normal sampling routine.	Module 07.
L_i , the incoming radiance	Sampling points on light sources directly, and building a 2D CDF over an environment map.	Modules 08 and 09.
Their product	No single strategy. Multiple importance sampling combines several strategies, each matching one factor, with weights that are robust to any one of them failing.	Module 08.

A BAD P IS WORSE THAN NO P AT ALL

Importance sampling is not uniformly beneficial. If p is small in a region where f is large, then on the rare occasions a sample lands there, f/p is enormous.

That is exactly the firefly of Module 05, and a badly chosen density can produce far higher variance than uniform sampling would have.

The rule: never let p approach zero where f might not be zero. If in doubt, mix a small uniform component into the density — 'defensive sampling'. The cost is a slight increase in variance everywhere; the benefit is removing an unbounded tail.

This is also why the MIS weights of Module 08 are designed for *robustness* rather than optimality. A strategy that is excellent 99% of the time and catastrophic 1% of the time is worse, in practice, than one that is merely good always.

3 Sample placement

	Independent uniform	Stratified
Method	Draw each sample independently.	Divide the domain into N equal cells; place one jittered sample in each.
Coverage	Clumps and gaps. Random points are not evenly spread — they are even only in expectation, and any particular set is not.	Guaranteed , with randomness retained within each cell so no aliasing is introduced.
Variance	The $1/N$ baseline.	Never worse, usually better , and dramatically better on integrands with structure.
Cost	—	Nothing. Index arithmetic.

Always stratify. It cannot hurt, it costs nothing, and the benefit on primary rays and light sampling is immediate and visible.

STRATIFICATION DOES NOT SURVIVE HIGH DIMENSIONS

Stratifying d dimensions with k divisions in each requires k^d samples. At $d = 10$ with a modest $k = 4$, that is over a million samples — for one pixel.

Latin hypercube sampling avoids the explosion: stratify each dimension *independently* into N strata, then randomly permute the assignment across dimensions. N samples suffice for any d , and every 1D projection is perfectly stratified.

The limitation is that it guarantees nothing about the *joint* distribution — the samples can still clump in 2D while being perfectly spread in each axis.

Padded sampling is the practical answer. Stratify carefully in the two to four dimensions that matter most — pixel position, lens position, the first bounce direction, the first light sample — and use cheaper, lower-quality samples deeper in the path, where the integrand is smoother and errors are less visible. Every production renderer does something of this kind.

3.1 Low-discrepancy sequences

Discrepancy measures how far a point set deviates from perfectly uniform coverage. Low-discrepancy sequences are deterministic constructions that achieve much better coverage than random points, and they are designed so that *any* prefix of the sequence is well distributed — which matters because you do not know in advance how many samples a pixel will need.

Sequence	Construction	In practice
Halton	Radical inverse in a different prime base per dimension.	Good to about ten dimensions; the high primes correlate badly without scrambling.
Sobol	Base 2 throughout, using direction numbers.	The practical default. Fast to generate, scrambles well, and handles high dimensions better than Halton.

Sequence	Construction	In practice
Hammersley	Halton with the first dimension replaced by i/N .	Slightly better, but requires N to be fixed in advance , which rules out adaptive sampling.
Blue noise	Point sets optimised so the error spectrum has little low-frequency energy.	See below.

On integrands of bounded variation these converge as $O(\log^d N / N)$, which is asymptotically far better than $1/\sqrt{N}$. **The caveat matters:** rendering integrands are discontinuous at every visibility boundary and do not have bounded variation, so the theoretical rate is not achieved. The practical improvement is nonetheless real and typically worth a factor of two to four in equal-quality time.

BLUE NOISE IMPROVES NOTHING MEASURABLE AND IS WORTH USING

Blue noise sample placement does not reduce RMSE. It **redistributes the error into high spatial frequencies**, leaving the total unchanged.

The human visual system is markedly less sensitive to high-frequency noise than to low-frequency blotching, so two images with identical RMSE can look very different — and the blue-noise one looks substantially cleaner.

Denoisers also prefer it, for the related reason that high-frequency noise is easier to separate from the underlying signal than low-frequency structure is. A denoiser applied to blue-noise input produces a better result from the same sample count.

This is a case where RMSE is the wrong metric and reporting it alone would conclude the technique does nothing. Perceptual metrics such as FLIP capture the difference; so does looking at the images.

4 Russian roulette

Paths must terminate. Truncating at a fixed depth is biased (Module 02) — it simply discards the tail of the Neumann series. Russian roulette terminates paths *stochastically* while leaving the expectation exactly correct.

$$F' = 0 \text{ with probability } q; \quad F / (1 - q) \text{ otherwise}$$

$$E[F'] = q \cdot 0 + (1 - q) \cdot F / (1 - q) = F$$

Paths are killed at random, and the survivors are scaled up by exactly the factor needed to compensate for those that died. The expected value is untouched; the variance increases, because some samples now return zero and others return more than they would have. **You are trading variance for speed**, which is usually a good trade because the time saved buys more samples.

HOW TO CHOOSE THE SURVIVAL PROBABILITY

Base it on the path's accumulated throughput. A path whose throughput has fallen to 0.01 can contribute little even if it survives, so it is cheap to terminate. A path still carrying 0.9 should be kept. Setting the survival probability to the throughput's maximum component is the standard choice.

Do not start before bounce three or four. Early bounces carry most of the energy in the image, and killing them increases variance substantially while saving little time.

Clamp the survival probability from below — 0.05 is a common floor. Otherwise the $1/(1-q)$ factor on a surviving path becomes enormous, and roulette starts *manufacturing* fireflies, which is the opposite of what you wanted.

Evaluate it at equal time, not equal samples. At a fixed sample count roulette always looks worse, because it is strictly adding variance. Its entire benefit is that each sample is cheaper, so you get more of them — which only shows up in an equal-time comparison. This mistake is extremely common.

5 The order to apply these

Step	Technique	Why in this position
1	Stratification	Free, never harmful, immediately visible on primary rays.
2	Cosine-weighted sampling	Free via Malley, and cancels a factor of the integrand exactly.
3	BRDF sampling (Module 07)	Essential the moment any surface is glossy.
4	Light sampling (Module 08)	Essential the moment any light is small.
5	MIS (Module 08)	The single largest win in the course. Needs 3 and 4 first.
6	Low-discrepancy sequences	A further factor of two to four, once the strategy is right.
7	Russian roulette	Speed at some variance cost. Last, because it interacts with everything above.

Measure after each step, at equal time. Several of these interact, and intuition about which will help is unreliable — the point of the convergence-plot script from Module 05 is that you do not have to guess.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
PBRT 4th ed. — Chapter 2.2–2.4 and Chapter 8, Sampling and Reconstruction	Inversion, the standard sampling routines, stratification, and the Sobol sampler with complete code.

Resource	What it covers for this module
PBRT — A.5, Sampling Algorithms	The reference list of how to sample every common distribution in rendering. Worth bookmarking.
Veach thesis — Chapter 2 on variance reduction	Careful treatment of importance sampling, including when it fails and why defensive sampling is sometimes necessary.
Christensen & Jarosz — The Path to Path-Traced Movies (free)	What production renderers actually do about sampling, stratification, and roulette. Useful reality check on §5.
Heitz — blue-noise sampling papers (free)	The perceptual argument of §3 developed properly, with sample sets you can use.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the inverse CDF method for an exponential distribution and verify with a χ^2 test against the analytic density.
- 2 Implement uniform and cosine-weighted hemisphere sampling. Verify both with χ^2 tests. Then render a diffuse scene with each and report the variance ratio.
- 3 Deliberately use u_1 instead of $\sqrt{u_1}$ in the disk radius. Plot the resulting sample distribution and explain what the furnace test reports.
- 4 Construct a case where importance sampling increases variance: choose a p that is small where f is large. Report the variance against uniform sampling.
- 5 Implement stratified sampling for pixel positions and measure the variance reduction on an edge-heavy scene against independent uniform sampling.
- 6 Implement a Sobol sampler and produce a convergence plot against independent random sampling on the same scene. Report both slopes.
- 7 Compare stratified, Latin hypercube, and Sobol sampling at 16, 64, and 256 samples per pixel. Report RMSE for all nine combinations.
- 8 Implement Russian roulette. Compare against a fixed bounce limit at **equal time**, and separately at equal sample count. Report both and explain the difference.
- 9 Apply the seven steps of §5 one at a time, measuring equal-time RMSE after each. Produce a single table. This is the most useful exercise in the module.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the inverse CDF method and explain geometrically why it works.
- 2 Describe Malley's method and show that the cosine and π cancel for a Lambertian surface.
- 3 What causes variance in a Monte Carlo estimator, and what would the ideal density be?
- 4 Why can the ideal density never be used, and what is matched instead?
- 5 How can importance sampling make an estimator worse, and what is the defence?
- 6 Why does stratification fail in high dimensions, and what two approaches replace it?
- 7 What does a low-discrepancy sequence guarantee, and why is the theoretical rate not achieved in rendering?
- 8 Why use blue noise if it does not reduce RMSE?
- 9 Prove Russian roulette is unbiased, and give three rules for applying it well.
- 10 Why must Russian roulette be evaluated at equal time rather than equal sample count?